SOFTWARE DESIGN DOCUMENT
for
Rendivo

2.0

13.12.2025

Prepared by

Bengisu Duru Göksu

Selin Bardakcı

Dicle Çoban

Zeynep Yiğit

Ahmet Can Hatipoğlu

TABLE OF CONTENTS

1 Introduction **4**
1.1 Purpose of the system 4
1.2 Design goals 4
1.3 Definitions, acronyms, and abbreviations 5
1.4 References 5
1.5 Overview 6
2 Current software architecture **7**
3 Proposed software architecture **7**
3.1 Overview 7
3.2 Subsystem decomposition 7
3.2.1 User Management Subsystem 9
3.2.2 Business Management Subsystem 10
3.2.3 Staff Management Subsystem 11
3.2.4 Appointment Management System 12
3.3 Hardware/software mapping 12
3.3.2 Web and Mobile Platform Mapping 13
3.3.4 Software Reuse 16
3.4 Persistent data management 16
3.4.1 Database Selection 16
3.4.2 Data Schema 17
Entity Relationships 17
3.4.3 Database Encapsulation 17
3.5 Access control and security 18
3.5.1 User Roles and Access Model 18
3.5.2 Access Control Matrix 18
3.5.3 Authentication and Authorization 19
3.6 Global software control 20
3.6.1 Control Architecture 20
3.6.2 Authentication and Authorization Control 20
3.6.3 Appointment Control Flow 20
3.6.4 State-Based Control 21
3.6.5 Integration with External and Asynchronous Services 22
3.4 Boundary conditions 22
3.7.1 Authentication and Authorization Boundaries 26
3.7.2 Data Integrity Constraints 26
3.7.3 Scheduling and Appointment Constraints 26
3.7.4 User and Business Constraints 26
3.7.5 Error Handling and Failure Conditions 26
3.7.6 Security Boundaries 27
3.7.7 System Limitations 27
4 Subsystem services **27**

4.1 User Management Subsystem 27
4.2 Business Management Subsystem 29
4.3 Staff Management Subsystem 30
4.4 Appointment & Scheduling Subsystem 32
4.5 Calendar & Realtime Sync Subsystem (Future Extension) 33
4.6 Notification Subsystem (Future Extension) 35
4.7 Search & Discovery Subsystem (Future Extension) 36
4.8 Admin Review & Platform Operations Subsystem (Future Extension) 38
5 Glossary of Terms 40
6 Traceability 42
6.1 Functional Requirements Traceability 42
6.2 Nonfunctional Requirements Traceability 44

1 Introduction

1.1 Purpose of the system

The purpose of the Rendivo system is to define and document the architectural design of a multi-tenant appointment management platform. This document describes the structural organization of the system, the major software components, and the design decisions that guide how data, control flow, and responsibilities are distributed across the system.

Rendivo is designed as a service-oriented backend system with clear separation between authentication, business logic, data access, and validation layers. The architecture supports multiple user roles operating within isolated business contexts, enabling secure and controlled access to shared infrastructure resources.

This document focuses on explaining how the system satisfies its design goals such as modularity, data consistency, role-based access control, and scalability by mapping architectural elements to the requirements defined in the Requirement Analysis Document. The purpose of this system design is to provide a clear and maintainable blueprint for implementation, future extension, and evaluation.

1.2 Design goals

The primary design goal of the Rendivo system is to provide a modular and maintainable software architecture that cleanly separates concerns between authentication, business logic, data access, and validation. This separation allows individual components to evolve independently, simplifies debugging and testing, and improves long-term maintainability of the system.

A key architectural goal is **multi-tenancy with strict data isolation**. The system is designed so that all core entities such as services, staff members, shifts, and appointments are scoped to a specific business context. This ensures that multiple businesses can operate on the same platform without data leakage or unauthorized access, while sharing the same underlying infrastructure.

Another important design goal is **data consistency and integrity**. The system relies on a relational data model to manage core transactional data such as appointments and schedules. Scheduling conflicts, including overlapping shifts and appointment collisions, are prevented through application-level validation rules that enforce consistency before data is persisted.

The architecture also prioritizes **role-based access control (RBAC)**. Different user roles (customers, staff members, and business owners) interact with the system through clearly defined permissions enforced at the API level. This design ensures that each role can only access and modify resources within its authorized scope.

Scalability and extensibility are additional design goals. The backend is structured to support future growth, including the integration of external services such as federated authentication providers and real-time data synchronization mechanisms. Design decisions favor extensible patterns over tightly coupled implementations, allowing new features to be added without major architectural changes.

Finally, the system is designed with **security and robustness** in mind. Stateless authentication using JSON Web Tokens, centralized validation mechanisms, and consistent authorization checks across endpoints contribute to a secure and reliable backend architecture suitable for production environments.

1.3 Definitions, acronyms, and abbreviations

Term	Definition
SDD	Software Design Document describing the architecture and design decisions of the system.
RBAC	Role-Based Access Control used to restrict system access based on user roles.
JWT	JSON Web Token used for stateless authentication between client and server.
Multi-Tenancy	An architectural approach where multiple businesses share the same system while keeping their data isolated.
Tenant	A business entity operating independently within the Rendivo platform.
Shift	A predefined time interval representing staff availability.
Appointment	A predefined time interval representing staff availability.
Availability	A predefined time interval representing staff availability.

1.4 References

● **[REF-01]** Rendivo Requirement Analysis Document, CSE443 – Software Engineering.

● **[REF-02]** IEEE 29148-2018 — Requirements Engineering – Use Case Diagrams.

● **[REF-03]** RFC 7519 — JSON Web Token (JWT).

● **[REF-04]** ISO 8601 — Date and Time Formats (timestamps used in scheduling and logging).

● **[REF-05]** Node.js (LTS) Documentation.

● **[REF-06]** Sequelize ORM Documentation.

- **[REF-07]** MySQL 8.0 Reference Manual (ACID properties, relational constraints).
- **[REF-08]** Firebase Authentication and Realtime Database Documentation.
- **[REF-09]** RESTful API Design Guidelines and Best Practices.
- **[REF-10]** Figma — Rendivo Web & Mobile UI (link: FIGMA LINK).

1.5 Overview

This Software Design Document defines the architectural structure and design decisions of the Rendivo system. The document is organized to first establish the current architectural context of the system, followed by a detailed presentation of the proposed target architecture, including subsystem responsibilities, deployment structure, persistent data handling, and security mechanisms.

Section 2 — Current software architecture describes the existing system state, architectural constraints, and design limitations that motivated the transition toward the proposed architecture.

Section 3 — Proposed software architecture presents the target architecture of the Rendivo system in a structured and layered manner:

- **Section 3.1 — Overview** provides a high-level architectural description of the system, outlining its main components and their interactions.
- **Section 3.2 — Subsystem decomposition** details the logical decomposition of the system into subsystems and components, clarifying responsibilities and boundaries.
- **Section 3.3 — Hardware/software mapping** describes how software components are deployed across infrastructure resources and execution environments.
- **Section 3.4 — Persistent data management** explains the data storage strategy, database organization, and data access mechanisms used by the system.
- **Section 3.5 — Access control and security** describes authentication, authorization, and data protection mechanisms enforced across the system.
- **Section 3.6 — Global software control** outlines system-wide control flows, request handling, and coordination mechanisms between components.
- **Section 3.7 — Boundary conditions** define system assumptions, constraints, and external dependencies that impact the architecture.

Section 4 — Subsystem services specifies the services exposed by each subsystem and their role within the overall system architecture.

Section 5 — Glossary of Terms defines domain-specific terminology and abbreviations used throughout the document.

Section 6 — Traceability establishes the mapping between architectural elements and system requirements, supporting design validation and verification.

2 Current software architecture

Prior to the Rendivo system, there was no unified, integrated software architecture supporting appointment management, staff scheduling, and business operations within a single platform. Existing solutions in this domain are typically fragmented, relying on a combination of manual processes, third-party tools, or loosely coupled applications that lack a consistent architectural foundation.

Commonly observed architectures for similar systems include monolithic web applications with tightly coupled business logic, limited role-based access control, and minimal separation between data access, application logic, and presentation layers. In many cases, appointment scheduling, staff management, and customer data are handled through separate tools or spreadsheets, resulting in data inconsistency, limited automation, and poor scalability.

From an architectural perspective, these approaches introduce several limitations. Authentication and authorization mechanisms are often simplistic or externally dependent, making it difficult to support multiple user roles such as customers, staff members, and business owners within a single system. Data persistence is frequently handled without clear transactional boundaries, increasing the risk of conflicting updates and integrity issues, especially in concurrent scheduling scenarios. Furthermore, availability checking and time-slot calculation are often implemented manually or client-side, leading to unreliable booking behavior.

The architectural assumptions identified during the requirements analysis highlight the need for a centralized backend system capable of enforcing business rules, managing persistent data consistently, and supporting secure, role-aware access to system resources. Additionally, modern deployment expectations require the system to be cloud-compatible, scalable, and capable of integrating external authentication providers when needed.

These limitations and assumptions form the basis for the proposed Rendivo architecture, which introduces a structured backend-driven design with clearly defined subsystems, centralized data management, and explicit security controls to address the shortcomings of existing approaches.

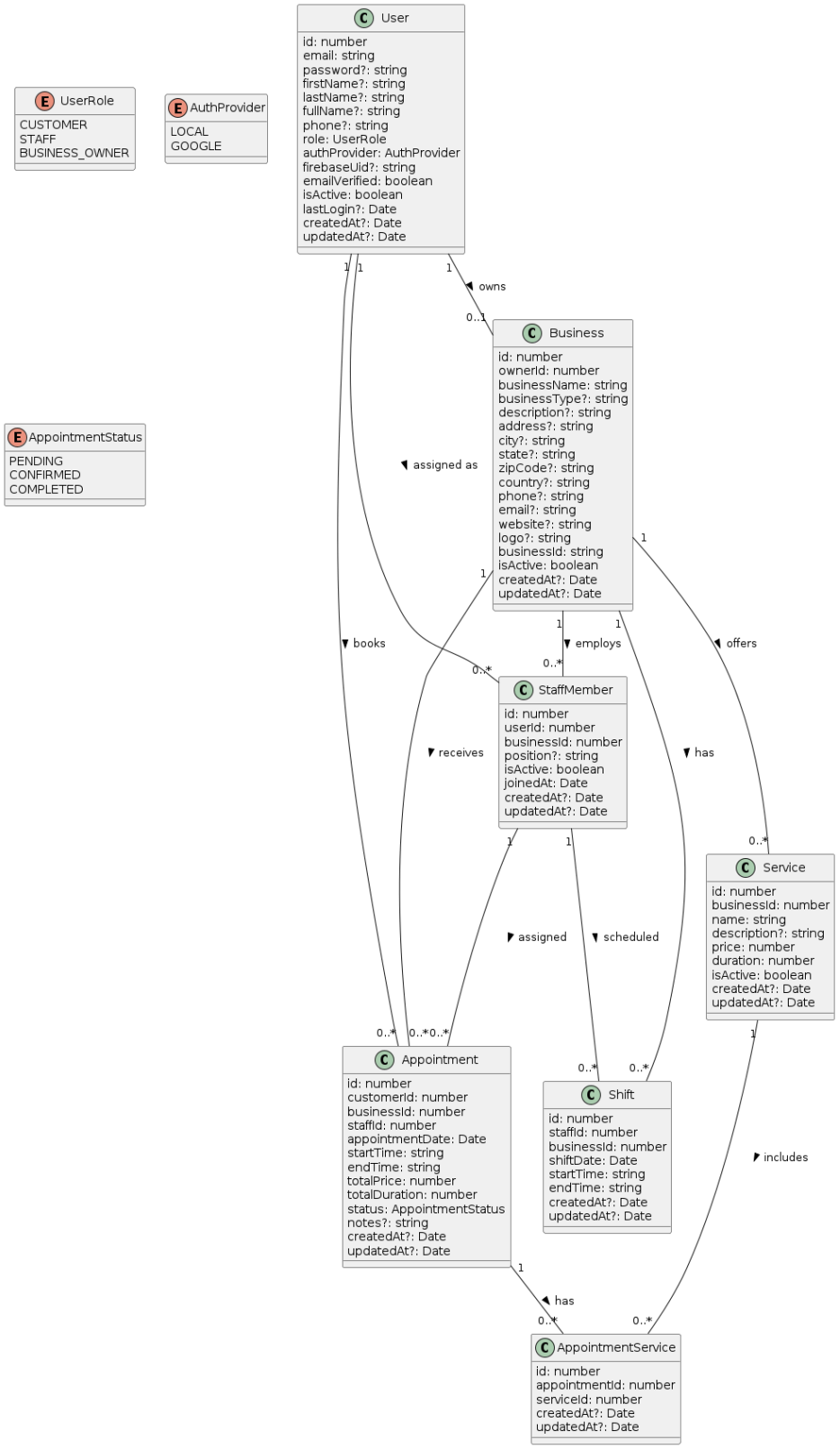
3 Proposed software architecture

3.1 Overview

3.2 Subsystem decomposition

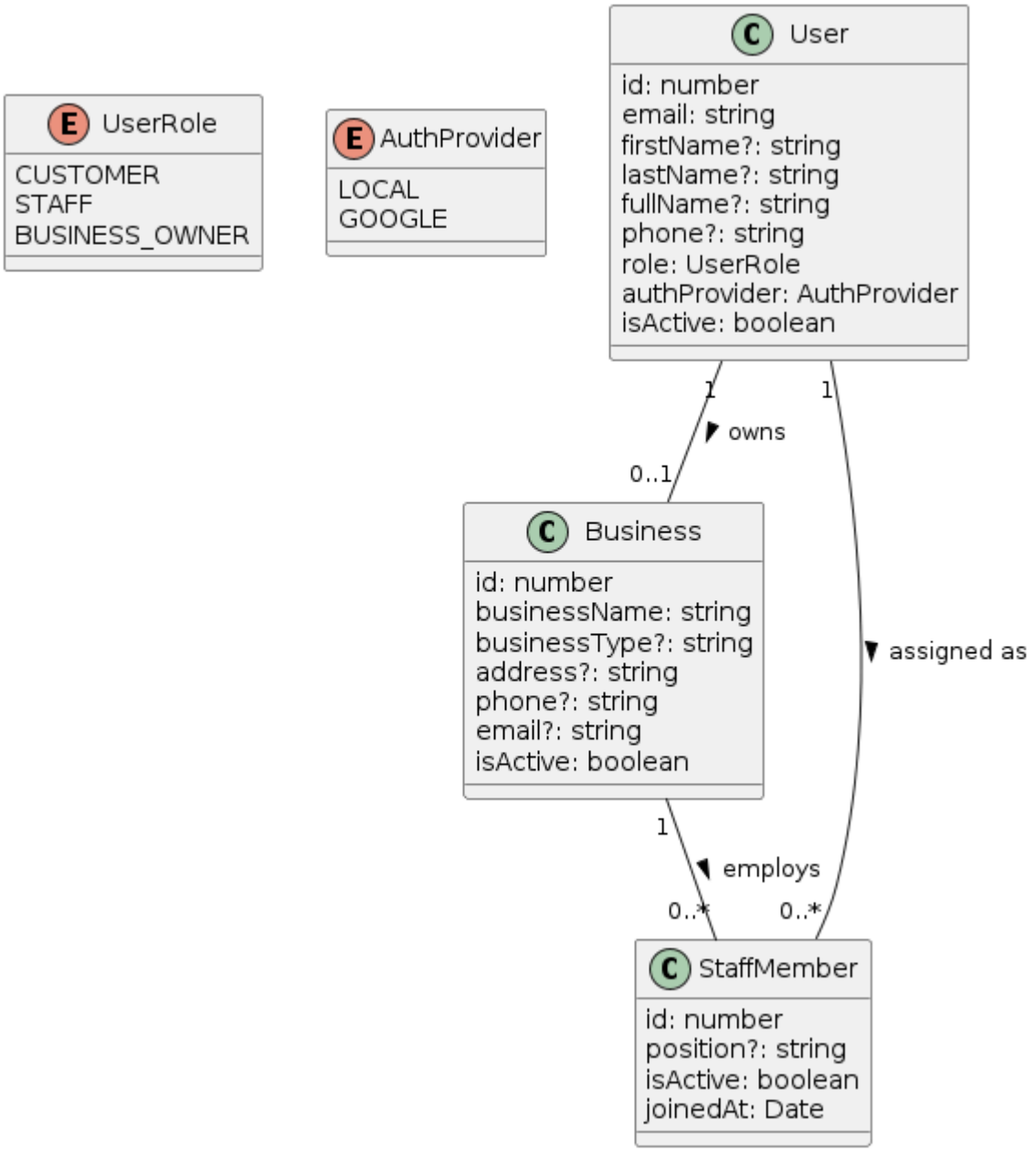
The Rendivo system is decomposed into a set of core domain subsystems that model the fundamental entities and relationships required for appointment-based service management. This decomposition focuses on the persistent business objects and their interactions, independent of presentation layers such as web or mobile clients. The main domain entities include `User`, `Business`, `StaffMember`, `Service`, `Appointment`, `AppointmentService`, and `Shift`. These classes capture user roles, business ownership, staff organization, service offerings, appointment scheduling, and working shifts. The

relationships among these entities define ownership, employment, assignment, and many-to-many service associations, forming the structural foundation of the system's backend architecture.



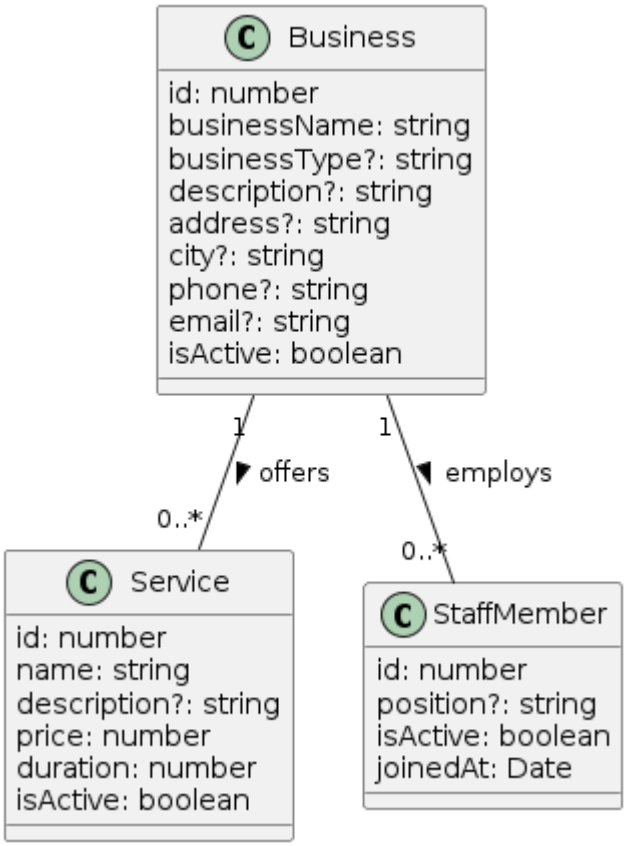
3.2.1 User Management Subsystem

The User Management Subsystem provides a centralized identity model for all actors interacting with the Rendivo platform. It defines the `User` entity, which encapsulates authentication-related attributes, role information, and account status. This subsystem supports multiple user roles and enables consistent authentication and role-based access control across both web and mobile clients, while delegating organizational and operational relationships to other subsystems.



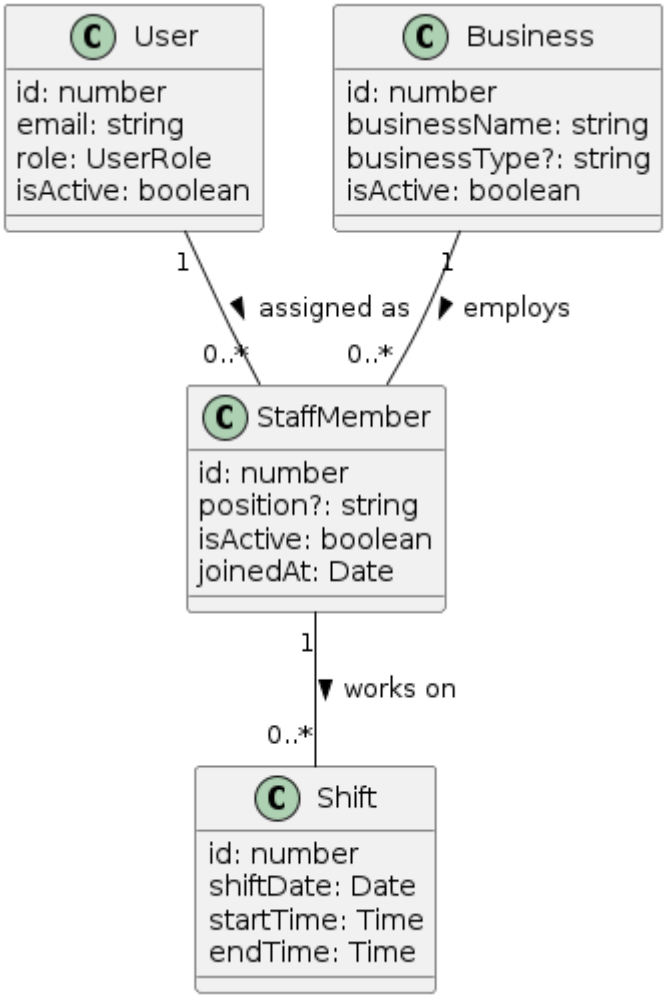
3.2.2 Business Management Subsystem

The Business Management Subsystem is responsible for representing service-providing organizations and their core operational structure within the Rendivo platform. The `Business` entity defines the organizational unit owned by a business owner and serves as the central context for services, staff members, and appointments. Services offered by a business are modeled through the `Service` entity, which captures pricing, duration, and availability information. This subsystem encapsulates business-level configuration and service definitions, providing the structural foundation required for scheduling and appointment management in higher-level subsystems.



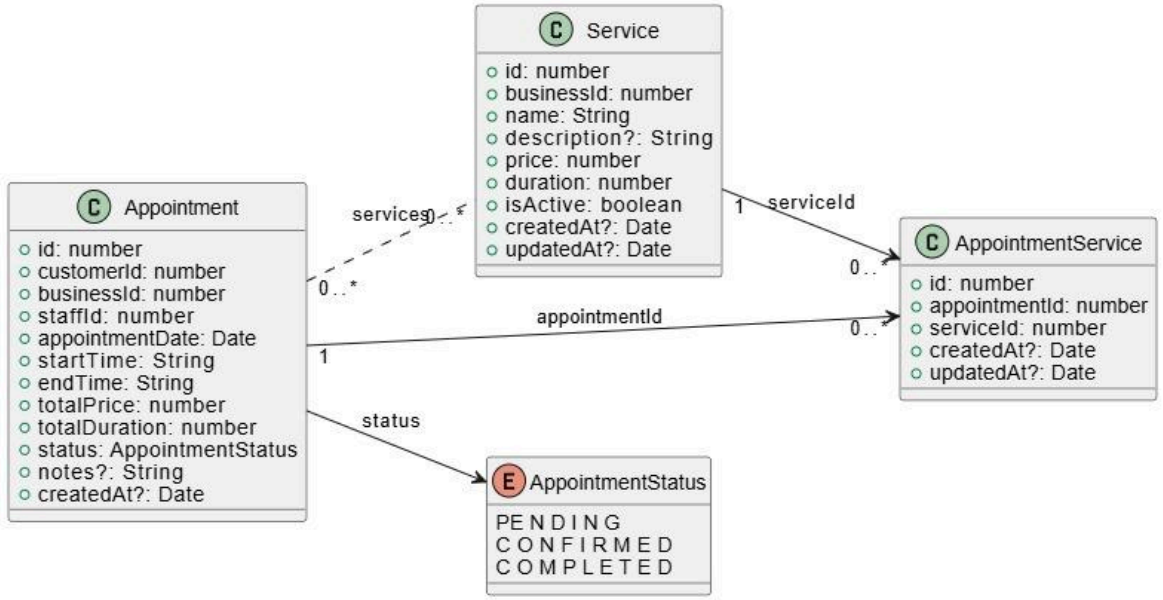
3.2.3 Staff Management Subsystem

The Staff Management Subsystem is responsible for representing employees working under a specific business and managing their association with both users and businesses. A `StaffMember` acts as a linking entity between `User` and `Business`, enabling role-based separation between business owners and operational staff. This subsystem supports staff assignment, shift scheduling, and appointment allocation while ensuring that each staff member belongs to exactly one business and is linked to a single user account. The design reflects the implemented data model and enforces business ownership, staff activation status, and temporal work constraints through shifts.



3.2.4 Appointment Management System

Appointment Management Subsystem is responsible for modeling and managing appointments between customers and businesses. An Appointment represents a scheduled interaction defined by a specific date, assigned staff member, duration and price. Appointments follow a state-based lifecycle by the AppointmentStatus, ensuring controlled transitions between pending, confirmed and completed states.
Relationship between appointments and services is realized through AppointmentService. which enables a many-to-many relations between Appointment and Service.



3.3 Hardware/software mapping

The Rendivo system is deployed using a distributed client–server architecture in which software subsystems are mapped to distinct hardware nodes and cloud-based services. The architecture supports both web and mobile platforms while relying on a shared backend and database infrastructure. This section describes how major subsystems are assigned to hardware components and identifies challenges introduced by multiple deployment nodes and software reuse.

3.3.1 Deployment Mapping

The system is deployed across the following primary hardware and infrastructure components:

- **Client Devices**
    - Web clients run in standard web browsers on desktop or laptop computers.
    - Mobile clients run as cross-platform Flutter applications on iOS and Android devices.

- Client devices are responsible for user interaction and communicate with the backend through HTTPS-based REST APIs.

- **Application Server**
  - The backend system is deployed on a cloud-hosted Node.js runtime.
  - It hosts the Express-based API, authentication and authorization logic, business rules, and notification services.
  - A single backend serves both web and mobile clients, ensuring consistent behavior across platforms.
- **Database Server**
  - Persistent data is stored in a managed MySQL database hosted on Aiven Cloud.
  - The database server maintains all business, user, appointment, and scheduling data with transactional guarantees.
- **External Services**
  - Firebase Cloud Messaging (FCM) is used exclusively for real-time push notifications.
  - Gmail SMTP is used for transactional email delivery such as confirmations and reminders.

Deployment UML diagrams illustrating the mapping of software components to these hardware nodes are provided in this section.

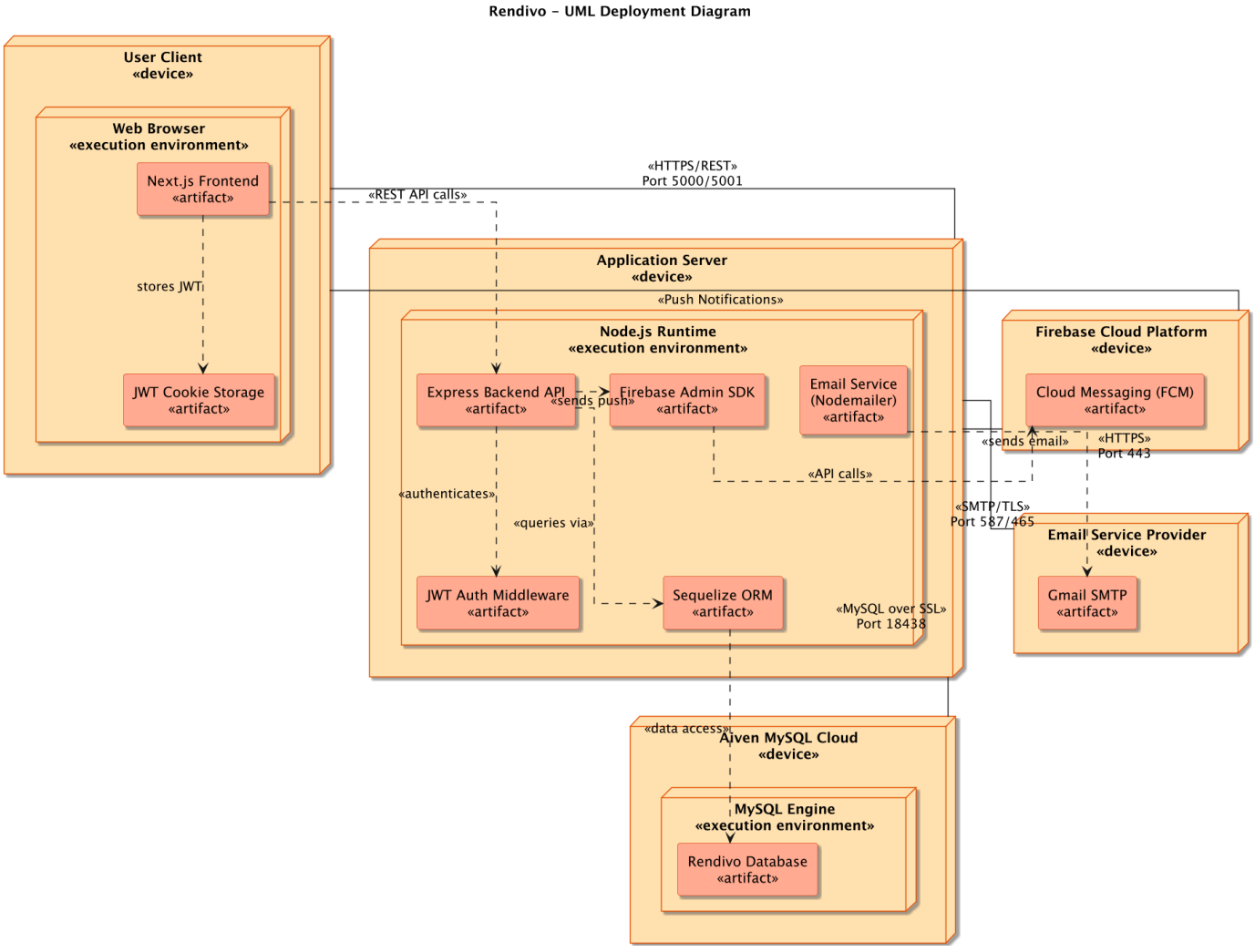
3.3.2 Web and Mobile Platform Mapping

- **Web Platform**
  - The web frontend executes within a browser environment and communicates with the backend via HTTPS.
  - Authentication tokens are managed at the client side and validated by the backend.
  - Database access is abstracted through an ORM-based data access layer.
- **Mobile Platform**

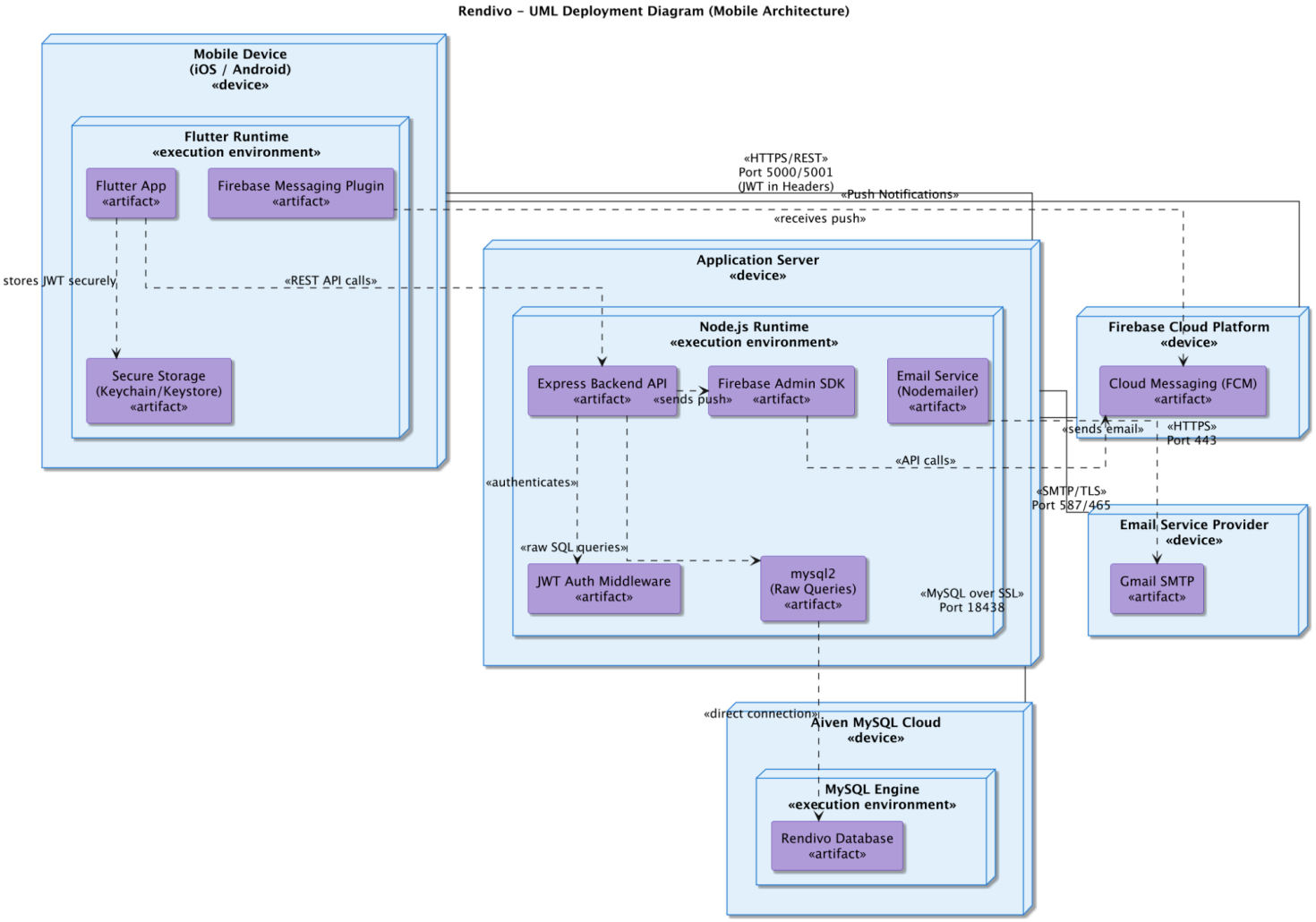
- The mobile application is implemented as a single Flutter codebase targeting both iOS and Android.
- Secure local storage mechanisms are used to protect authentication tokens.
- The mobile backend interacts with the same application server and database, using direct database access mechanisms where required.

Both platforms share the same backend services and database schema, enabling unified data management and consistent business logic.

UML Deployment Diagram of Rendivo Web App



UML Deployment Diagram Rendivo Mobile App



3.3.3 Multi-Node Issues and Mitigation

The use of multiple hardware and cloud nodes introduces several challenges:

- **Network Latency and Reliability**
  Mitigated through stateless API design, retry mechanisms, and encrypted communication channels.
- **Data Consistency**
  Ensured through ACID-compliant database transactions and centralized persistence management.
- **Authentication Across Nodes**
  Addressed using stateless JWT-based authentication, allowing requests to be validated independently across distributed components.
- **External Service Dependencies**
  Non-critical services such as notifications and emails are designed to fail gracefully without impacting core appointment operations.

3.3.4 Software Reuse

Rendivo incorporates off-the-shelf frameworks, libraries, and managed services to reduce development effort and increase system reliability. Software reuse introduces benefits such as faster development and proven stability, while also requiring careful integration and configuration management to maintain consistency across platforms.

The architecture isolates reused components behind well-defined interfaces, minimizing coupling and allowing components to be replaced or extended with minimal impact on the overall system.

3.4 Persistent data management

The Rendivo system manages persistent data required for appointment booking, business operations, and user management using a centralized relational database architecture. Persistent data includes users, businesses, staff members, services, appointments, shifts, and their relationships. The data management infrastructure is designed to ensure data consistency, integrity, and tenant-level isolation across the system.

The backend architecture relies on a relational database accessed through controlled data access layers, ensuring that business logic and data storage concerns remain properly separated.

3.4.1 Database Selection

MySQL 8.0 is selected as the primary database management system for the Rendivo platform due to the following reasons:

- **ACID Compliance**: Guarantees transactional integrity for critical operations such as appointment creation, modification, and cancellation.
- **Relational Model Suitability**: The system domain naturally maps to relational structures with well-defined entities and foreign key relationships.
- **Scalability and Reliability**: Cloud-hosted MySQL infrastructure provides automated backups and scalable resource management.
- **Security**: Encrypted connections protect data transmitted between application components and the database.

This selection supports reliable multi-tenant operation while maintaining strong data consistency guarantees.

3.4.2 Data Schema

The persistent data model is organized around a normalized relational schema consisting of the following core entities:

- **Users**: Stores account and identity information for all system actors, including customers, staff members, and business owners.
- **Businesses**: Represents service provider organizations operating within the system.
- **Staff Members**: Associates users with businesses in a service-providing role.
- **Services**: Defines business-specific offerings that can be booked through appointments.
- **Appointments**: Stores booking records linking customers, staff members, businesses, and scheduled time intervals.
- **Appointment Services**: A junction entity supporting many-to-many relationships between appointments and services.
- **Shifts**: Defines staff availability schedules used to determine valid booking intervals.

Entity Relationships

The schema enforces the following relationships:

- A business is owned by a single business owner.
- A business may employ multiple staff members.
- A business may offer multiple services.
- A customer may create multiple appointments.
- A staff member may be assigned to multiple appointments and shifts.
- Appointments may include one or more services through a junction relationship.

Foreign key constraints and relational cardinalities enforce referential integrity and prevent inconsistent data states.

3.4.3 Database Encapsulation

Access to persistent data is encapsulated through dedicated data access layers that isolate application logic from direct database manipulation. All data operations are performed through controlled interfaces that enforce validation, authorization, and consistency rules before persistence.

The encapsulation strategy ensures that:

- Application components do not directly manipulate database tables.
- Data integrity rules are consistently enforced across all access paths.
- Transactional operations maintain atomicity and consistency during concurrent access.
- The underlying database technology can be modified or extended without impacting higher-level system components.

This approach improves maintainability, security, and long-term extensibility of the system's data management architecture.

3.5 Access control and security

The Rendivo system enforces access control through a Role-Based Access Control (RBAC) model combined with token-based authentication mechanisms. This section describes the user model, access permissions across system resources, authentication mechanisms, and security controls applied to protect system data and operations.

3.5.1 User Roles and Access Model

Rendivo defines three primary user roles, each with clearly scoped responsibilities:

- **Customer**: End users who search businesses and services, create and manage their own appointments, and maintain personal profiles.
- **Staff**: Service providers employed by a business who manage their availability, view assigned appointments, and access business schedules.
- **Business Owner**: Administrative users responsible for managing business profiles, staff members, service offerings, shifts, and appointments.

Access permissions are strictly scoped based on role and ownership rules to prevent unauthorized access to system resources.

3.5.2 Access Control Matrix

Access to system resources is regulated through an access matrix that defines allowed operations per role:

- **Users**: All roles may create and manage their own accounts and profiles. Access to other users' profiles is denied.
- **Businesses**: Customers may view business listings and details. Staff and business owners may view their own business details. Business creation and updates are

restricted to business owners.

- **Services**: Customers may view services. Service creation, modification, and deletion are restricted to business owners.
- **Appointments**: Customers may create, view, modify, and cancel their own appointments. Staff may view and update the status of assigned appointments. Business owners may manage all appointments within their business.
- **Shifts**: Customers may query public availability. Staff may view assigned shifts. Shift management is restricted to business owners.
- **Staff Management**: Staff listings are visible based on business context, while staff creation and removal are restricted to business owners.

This access matrix ensures least-privilege access and enforces tenant-level isolation.

3.5.3 Authentication and Authorization

The system uses token-based authentication as the primary access mechanism. Upon successful login, authenticated users receive a JSON Web Token (JWT) containing identity and role information. Tokens are transmitted with each request and validated by backend middleware before request processing.

Authorization is enforced through role-based checks applied at the API level. Requests that do not satisfy authentication or authorization requirements are rejected before reaching business logic.

The architecture supports extensibility for federated authentication providers through OAuth-based mechanisms, allowing future integration without changes to core authorization logic.

3.5.4 Security Measures

Rendivo applies multiple security controls to protect data and system integrity:

- **Data in Transit**: All client-server communication is protected using HTTPS.
- **Data at Rest**: User credentials are securely hashed before storage. Sensitive configuration values such as secrets and credentials are managed through environment-based configuration.
- **Key Management**: Authentication secrets and database credentials are centrally managed and excluded from source control.
- **Input Validation**: Request payloads are validated at both application and persistence layers to prevent malformed input and injection attacks.

These measures collectively ensure confidentiality, integrity, and availability of system resources.

3.6 Global software control

This section describes the overall control strategy of the Rendivo appointment management system and explains how control flows through the system components during runtime.

3.6.1 Control Architecture

Rendivo confirms a **layered, service-oriented** control architecture based on a **request-response** model. Web and mobile interact with backend systems through RESTful API endpoints. Each request is processed through a sequence of well-defined layers that ensure security, consistency, and separation of concerns.

Main Control Layers:

- **Presentation Layer:** Web and mobile clients responsible for collecting user input and initiating HTTP requests.
- **Authentication and Authorization Layer**: Middleware that confirms JWT and executes role-based access control.
- **Controller Layer**: Handles requests and processes to the appropriate domain logic
- **Domain Logic Layer:** Implement core business rules such as appointment creation, registration and staff-business association.
- **Persistence Layer:** Manages data storage and retrieval models to a relational database.

3.6.2 Authentication and Authorization Control

Global access control is applied using **token-based authentication**. For each request, the system follows these steps:

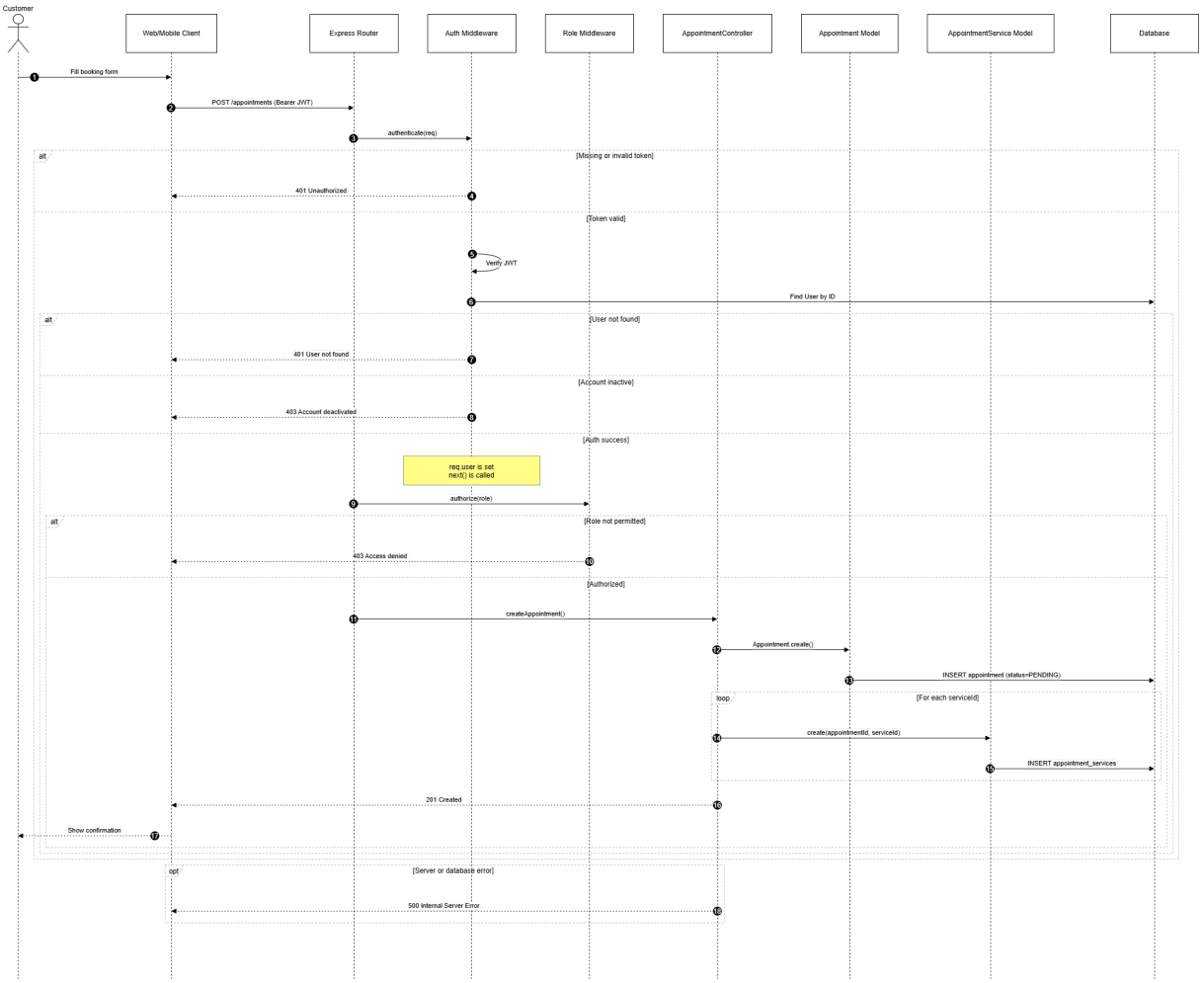
1. Extracts JWT from request header
2. Verifies token's validity and expiration
3. Retrieves the corresponding user entity from the database.
4. Checks whether the user account is active
5. Attaches the authenticated user context to the request.

Role-based authorization is applied to restrict sensitive operations to user roles (customer, staff, business owner). Requests that fail authentication or authorization checks are rejected before reaching the controller layer.

3.6.3 Appointment Control Flow

When a customer initiates an appointment creation request, the following sequence occurs:

1. Client sends a request to the appointment endpoint.
2. Authentication and authorization middleware validate the request.
3. The appointment controller processes the request and calls the appointment logic.
4. A new appointment entity is created with required attributes such as **customer, business, staff member of business, time interval, duration and price.**
5. The appointment is initially assigned a **pending** status.
6. Associated services are linked to the appointment.
7. The appointment data persisted in the database.



3.6.4 State-Based Control

Appointments follow a state-driven lifecycle that controls allowed system operations. The defined states:

- Pending
- Confirmed
- Completed
- Cancelled

State transitions are controlled by the system to prevent invalid operations. For example, a completed appointment cannot be cancelled, and only specific roles may trigger certain transitions. This approach improves data consistency and enforces business rules at the system level.

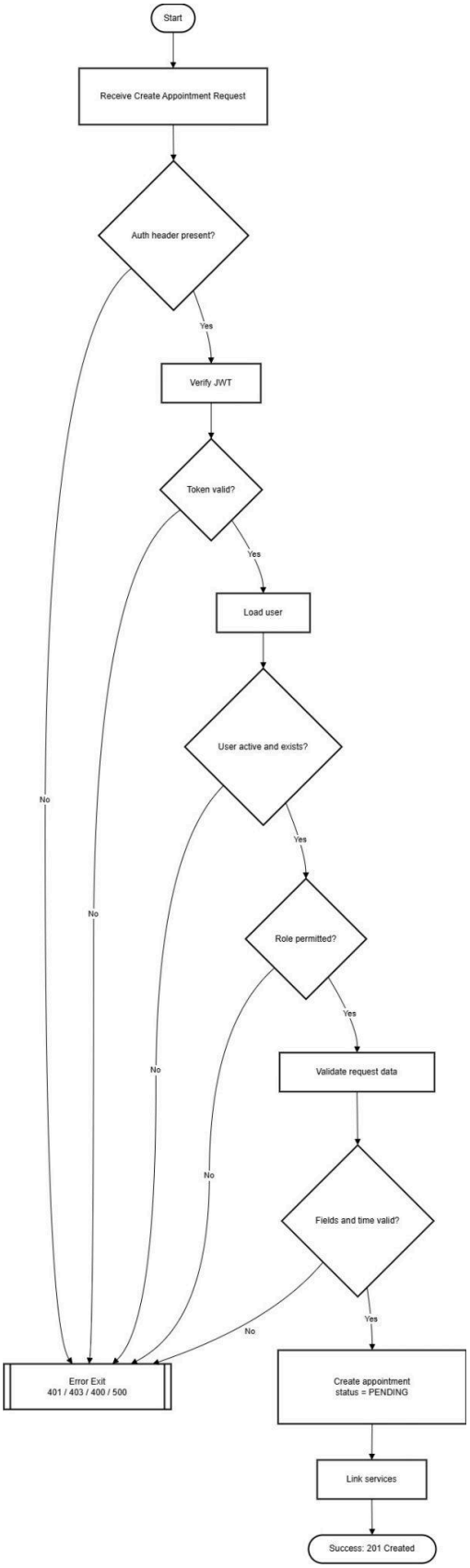
3.6.5 Integration with External and Asynchronous Services

The system supports multiple authentication providers, including local authentication and Firebase-based authentication. These external services are integrated into the global control flow without disrupting the core appointment logic.

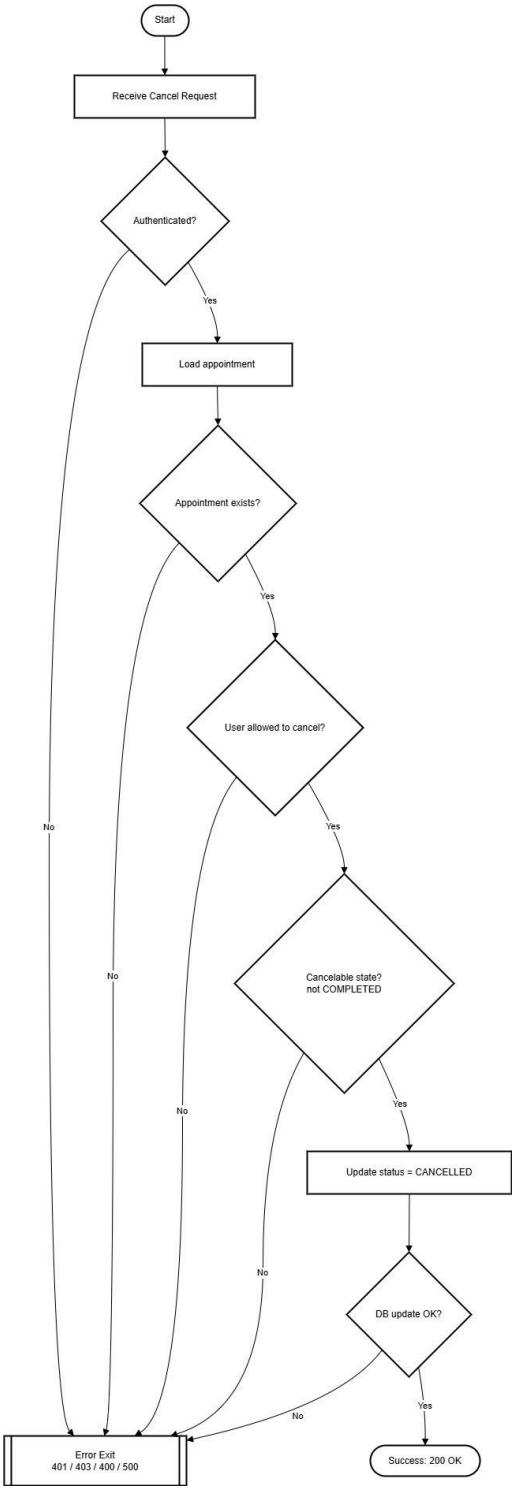
Certain operations, such as notification delivery or reminder scheduling, may be handled asynchronously to avoid blocking the primary request–response cycle. Asynchronous failures do not invalidate the core appointment transaction but are logged for monitoring and recovery purposes.

3.4 Boundary conditions

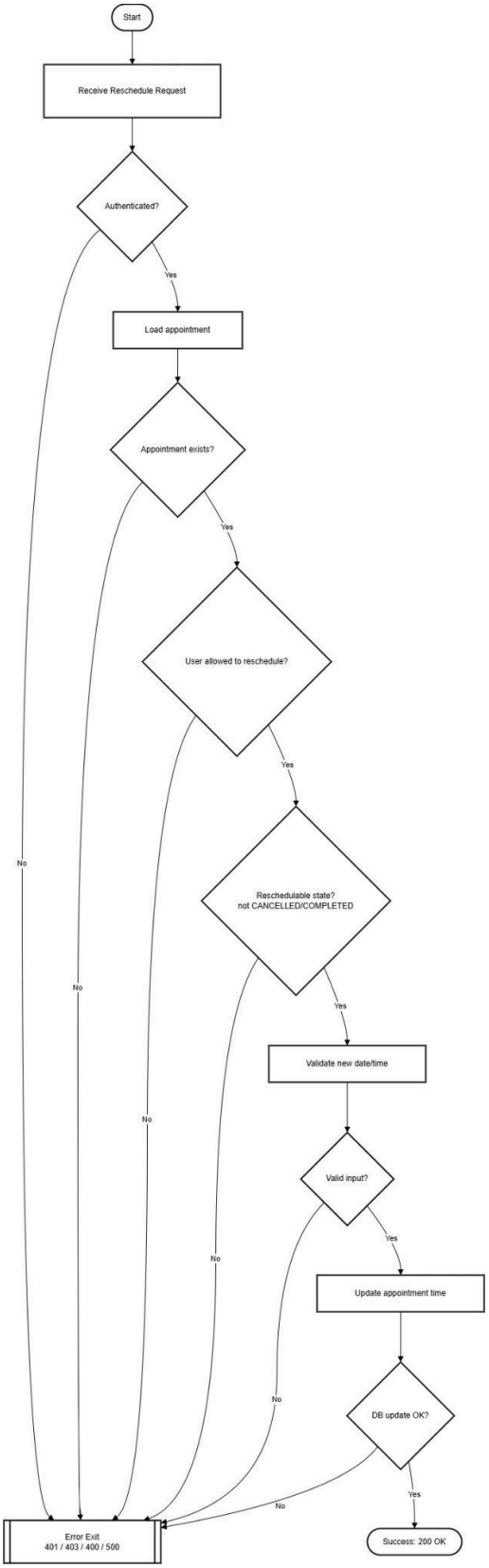
This section defines the operational limits, exceptional cases, and constraints under which the Rendivo system operates.



Activity Diagram for Create Appointment



Activity Diagram for Cancel Appointment



Activity Diagram for Reschedule Appointment

3.7.1 Authentication and Authorization Boundaries

- Requests without a valid authentication token are rejected.
- Expired or invalid tokens result in authentication failure.
- Users with deactivated accounts are denied access.
- Requests attempting operations outside the user's assigned role are rejected.

These boundaries ensure that only authorized users can access protected system functionality.

3.7.2 Data Integrity Constraints

The system enforces strict data integrity rules:

- Every appointment must reference an existing customer, business, and staff member.
- Mandatory fields such as appointment date, start time, end time, duration, and price must be provided.
- Appointment status values are restricted to a predefined enumeration.
- Referential integrity is enforced through **database-level foreign key** constraints.

Invalid or incomplete data submissions are rejected to preserve system consistency.

3.7.3 Scheduling and Appointment Constraints

- Appointments cannot be created without an assigned staff member.
- Appointment durations must be positive values.
- Time intervals must be explicitly defined.
- Inactive services or staff members cannot be used for appointment scheduling.

These constraints prevent logically inconsistent or invalid appointment records.

3.7.4 User and Business Constraints

- User email addresses must be unique across the system.
- Staff registration requires a valid business identifier.
- Business identifiers are system-generated and guaranteed to be unique.
- Inactive businesses and users are excluded from operational workflows.

3.7.5 Error Handling and Failure Conditions

The system handles exceptional conditions gracefully by returning appropriate error responses in cases such as:

- Duplicate registration attempts
- Invalid authentication credentials
- Missing or malformed request data
- Unauthorized access attempts
- Internal server or database failures

Clear error handling ensures robustness and improves system reliability.

3.7.6 Security Boundaries

- User passwords are never stored in plaintext and are securely hashed prior to persistence.
- Sensitive fields are excluded from API responses.
- Authentication tokens have limited validity periods to reduce security risks.

3.7.7 System Limitations

- The system assumes continuous availability of the database service.
- Logout operations are stateless and handled client-side unless token revocation mechanisms are introduced.
- Concurrent appointment conflicts are mitigated through persistence-layer constraints and controlled state transitions.

4 Subsystem services

4.1 User Management Subsystem

Responsibility:
Provides a centralized identity and authentication layer for all Rendivo users (**Customer, Staff, Business, Admin**) using a unified **User** entity. It exposes registration and login endpoints via **AuthController** and enforces RBAC through **AuthMiddleware** (`authenticate` / `authorize`).

Provided Services / Operations

- **AuthController.registerCustomer(req): Response**
  Creates a new **User** with `role = CUSTOMER` and `authProvider = LOCAL` (or as provided), stores credentials, and initializes `emailVerified` and `isActive`.

- **AuthController.registerStaff(req): Response**
  Creates a new **User** with `role = STAFF` (and any required staff onboarding fields handled in the staff subsystem/business linkage).

- **AuthController.registerBusiness(req): Response**
  Creates a new **User** with `role = BUSINESS` for the business owner/account (business entity creation is handled in the Business subsystem).

- **AuthController.login(req): Response**
  Authenticates a user (LOCAL credentials) and returns an authentication response (e.g., JWT/session payload).

- **AuthController.firebaseAuth(req): Response**
  Authenticates via external provider identity (Firebase) and maps/creates the user based on `firebaseUid` and `authProvider`.

- **AuthController.getProfile(req): Response**
  Returns the authenticated user's profile (safe fields; uses `toSafeObject()`).

- **AuthController.logout(req): Response**
  Performs logout behavior as implemented (commonly client-side token discard; server may respond successfully without state).

- **AuthMiddleware.authenticate(req, res, next): void**
  Validates JWT, loads the corresponding **User**, checks `isActive`, and attaches user context to the request.

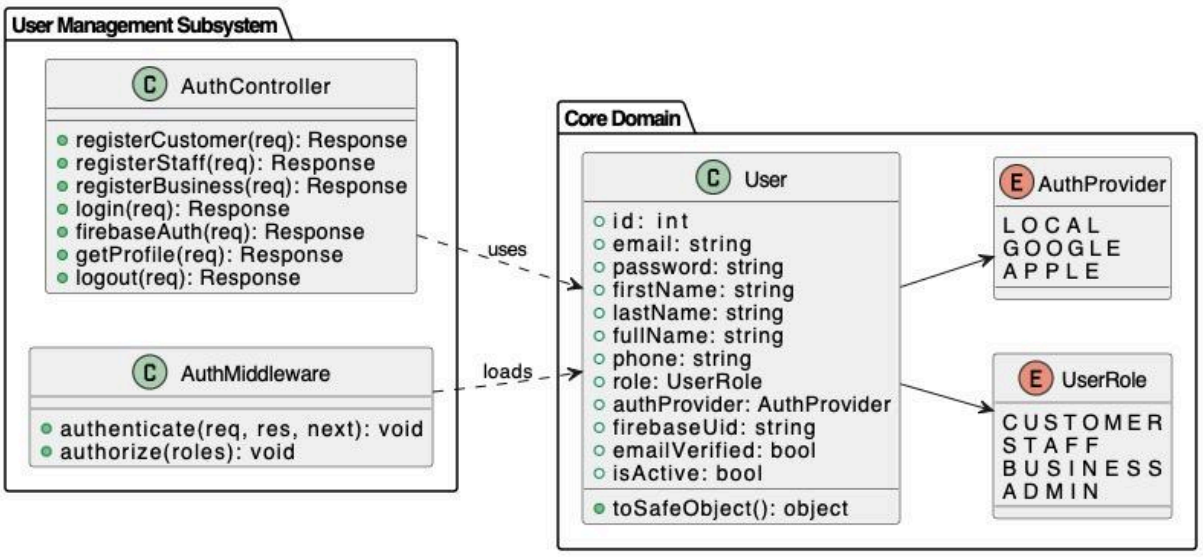
- **AuthMiddleware.authorize(roles): void**
  Enforces role-based access control by allowing only specified `UserRole` values.

Pre/Postconditions & Error cases:

- **Pre:**
  - Registration endpoints require mandatory fields (email/password and role-specific fields).
  - `login/firebaseAuth` requires valid credentials/provider token.
  - Protected endpoints require a valid JWT; user must be `isActive = true`.

- **Post:**
  - Successful registration creates a **User** with appropriate `UserRole` and `AuthProvider`.
  - Successful login/firebaseAuth returns a valid authentication response; `getProfile` returns a safe user object.

- **Errors:**
  `DuplicateEmail`, `InvalidCredentials`, `Unauthorized` / `TokenExpiredOrInvalid`, `AccountInactive`, `ProviderAuthFailed` *(for firebaseAuth)*.



4.2 Business Management Subsystem

Responsibility:
Handles **tenant (Business) onboarding** and establishes the **Business–Owner (User)** relationship in the core domain. This subsystem exposes registration operations that create a Business record (scoped by `businessId`) and supports staff onboarding through business association.

Provided Services / Operations

- **AuthController.registerBusiness(req): Response**
  Creates a new **Business (tenant)** owned by the requesting/identified **User** (`ownerId`) and persists the business profile data (e.g., name, contact, address fields). Generates/returns a unique `businessId` as the tenant identifier.

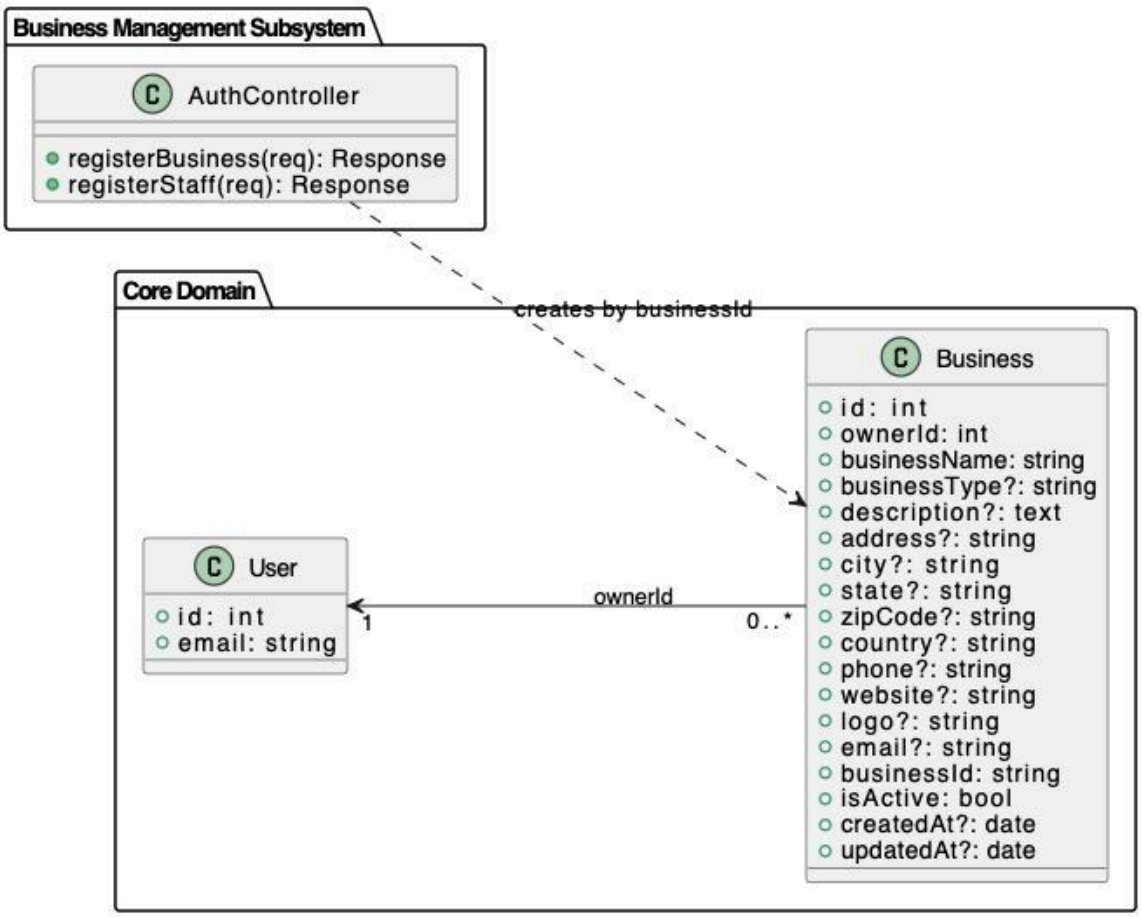
- **AuthController.registerStaff(req): Response**
  Registers a **Staff** account and associates it with the target business using the provided `businessId` (or business identifier in the request).

Pre/Postconditions & Error cases:

- **Pre:**
  - `registerBusiness`: request must contain valid business profile fields; the owner User must exist and be authenticated/authorized.
  - `registerStaff`: request must include a valid `businessId`; staff registration fields must be valid.

- **Post:**
  - `registerBusiness`: a Business entity is created with `ownerId` set; `businessId` is issued and returned; `createdAt/updatedAt` are set; `isActive` initialized per policy.
  - `registerStaff`: staff user is created and linked to the specified business context.

- **Errors:**
  `Unauthorized`, `InvalidRequestData`, `DuplicateBusinessId` (if applicable), `BusinessNotFound` (for staff registration), `BusinessInactive`, `Conflict/DuplicateRegistration`.



4.3 Staff Management Subsystem

Responsibility:
Manages **staff onboarding and membership** within a specific business tenant by creating and maintaining the **StaffMember** entity linked to a **User** and a **Business**. It also supports staff authentication and profile retrieval within the staff context.

Provided Services / Operations

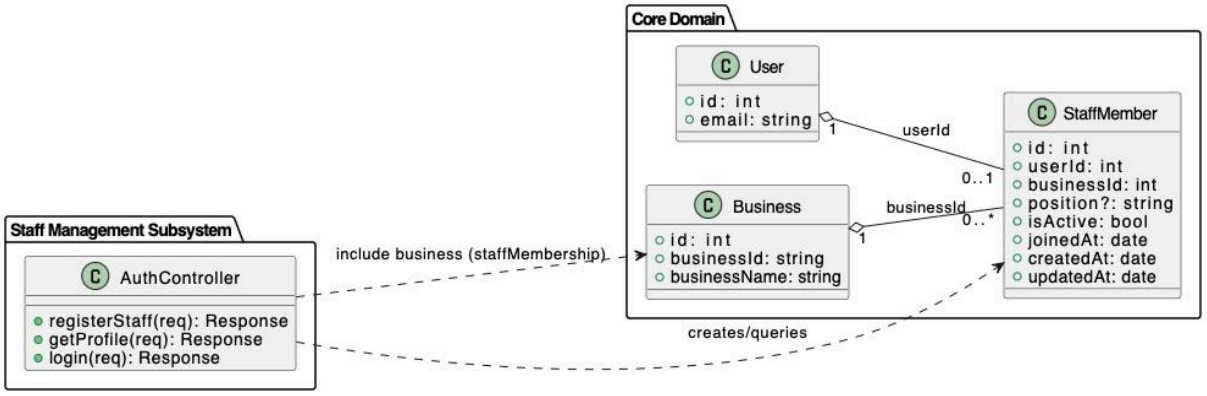
- **AuthController.registerStaff(req): Response**
  Creates a staff **User** (if not already existing per policy) and creates a **StaffMember** record that links `userId` ↔ `businessId` (staff membership). Initializes staff attributes such as `position`, `isActive`, and timestamps.

- **AuthController.login(req): Response**
  Authenticates a staff user and returns an authentication response (e.g., JWT/session info) for staff access.

- **AuthController.getProfile(req): Response**
  Returns the authenticated staff user's profile, including the associated **StaffMember** membership information (business linkage).

Pre/Postconditions & Error cases:

- **Pre:**
  - `registerStaff`: request must include a valid `businessId` (or business identifier used in your API) and valid staff registration fields; business must exist.
  - `login`: credentials/provider token must be valid; account must be active.
  - `getProfile`: request must be authenticated (valid token).
- **Post:**
  - `registerStaff`: a **User** is created/validated and a **StaffMember** record is created with `userId` and `businessId`; membership becomes queryable immediately.
  - `login`: successful authentication returns a valid token/session response.
  - `getProfile`: returns current user + staff membership details.
- **Errors:**
  `BusinessNotFound`, `InvalidBusinessId`, `DuplicateEmail` / `StaffAlreadyExists` *(if enforced)*, `InvalidCredentials`, `Unauthorized` / `TokenExpired`, `StaffInactive`.



4.4 Appointment & Scheduling Subsystem

Responsibility:
Manages the **appointment domain objects and lifecycle** by creating, updating, and retrieving **Appointment** records and their associated **services** through **AppointmentService**. It enforces valid **AppointmentStatus** transitions (Pending → Confirmed → Completed) and maintains consistency between Appointment and its service links.

Provided Services / Operations

- **createAppointment(req): Response**
  Creates an **Appointment** with `customerId`, `businessId`, `staffId`, `appointmentDate`, `startTime`, `endTime`, `totalPrice`, `totalDuration`, and initial `status = PENDING`. Also creates **AppointmentService** rows for each selected `serviceId`.

- **getAppointment(appointmentId): Response**
  Returns Appointment details **including linked services** (via AppointmentService → Service).

- **listAppointments(scope, filters): Response**
  Lists appointments for a given scope (e.g., by `customerId`, `staffId`, or `businessId`) with optional filtering (date range, status).

- **modifyAppointment(appointmentId, patchData): Response**
  Updates mutable appointment fields (e.g., time/date/notes) and recalculates totals if service links change.

- **updateAppointmentServices(appointmentId, serviceIds): Response** *(optional but recommended based on diagram)*
  Replaces or adjusts the AppointmentService links for an appointment (add/remove services).

- **transitionAppointmentStatus(appointmentId, targetStatus): Response**
  Applies allowed status transitions: **PENDING → CONFIRMED → COMPLETED** (and rejects invalid transitions).

- **cancelAppointment(...)** *(not in the shown diagram; include only if your model supports it)*
  If cancellation is supported later, it should set `status` to a cancelled state (requires adding `CANCELLED` to AppointmentStatus).

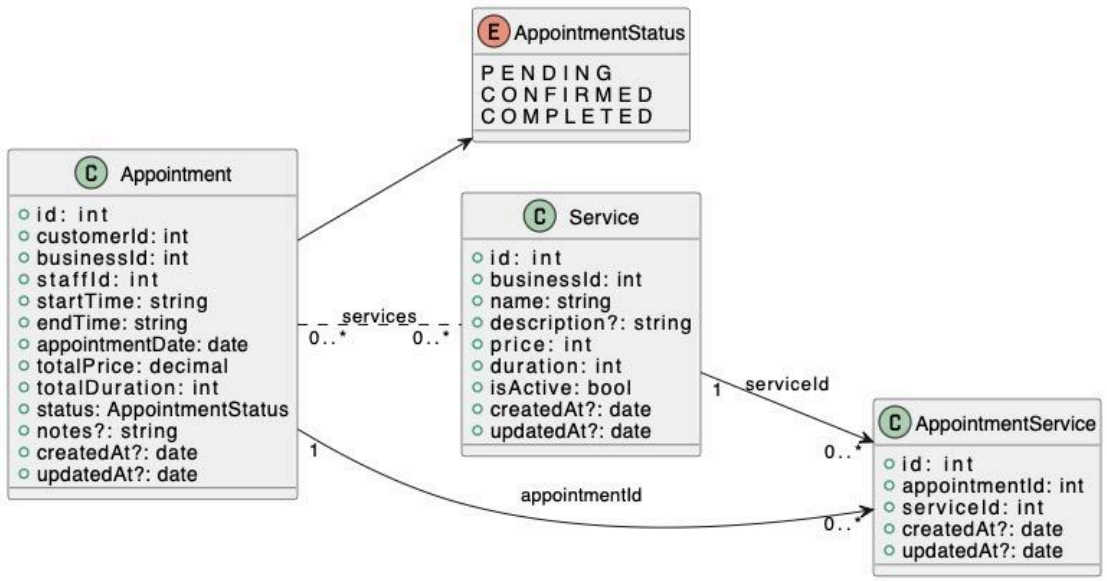
Pre/Postconditions & Error cases:

- **Pre:**
  - For create/modify: referenced `serviceId` values must exist; `appointmentDate/startTime/endTime` must be valid; `businessId/staffId/customerId` must be valid IDs.

- For status transitions: current status must allow the requested `targetStatus`.

- **Post:**
  - `createAppointment`: Appointment is persisted and **AppointmentService** rows are created; `totalPrice/totalDuration` reflect the linked services.
  - `modifyAppointment/updateAppointmentServices`: Appointment and its service links remain consistent; timestamps updated.
  - `transitionAppointmentStatus`: status updated to the target state and persisted.

- **Errors:**
  `AppointmentNotFound`, `InvalidTimeInterval`, `ServiceNotFound`, `InvalidStatusTransition`, `InvalidRequestData`, `UnauthorizedTenantAccess`.



4.5 Calendar & Realtime Sync Subsystem (Future Extension)

Responsibility:
Provides calendar views (day/week/month) and propagates schedule changes in real time to connected clients. Integrates with Firebase Realtime Database while keeping the backend as the source of truth.

Provided Services / Operations

- **CalendarController.getCalendarView(req): Response**
  Returns calendar blocks for UI rendering based on scope (businessId | staffId | customerId), viewType (day/week/month), and dateRange. Aggregates appointments, availability blocks, and relevant schedule constraints into a normalized structure for the frontend.

- **RealtimeSyncService.publishScheduleUpdate(eventType, entityId, payload): void**
  Publishes schedule events (e.g., AppointmentCreated/Updated/Cancelled, AvailabilityChanged) to tenant-scoped realtime channels. Ensures payload schema consistency and includes minimal metadata for client-side reconciliation.

- **RealtimeSyncService.subscribeChannelMapping(tenantId): ChannelMap**
  Maintains tenant-scoped channel mapping/keys (e.g., `/tenants/{tenantId}/business/{businessId}`, `/staff/{staffId}`, `/customers/{customerId}`) to prevent cross-tenant leakage and standardize subscription paths.

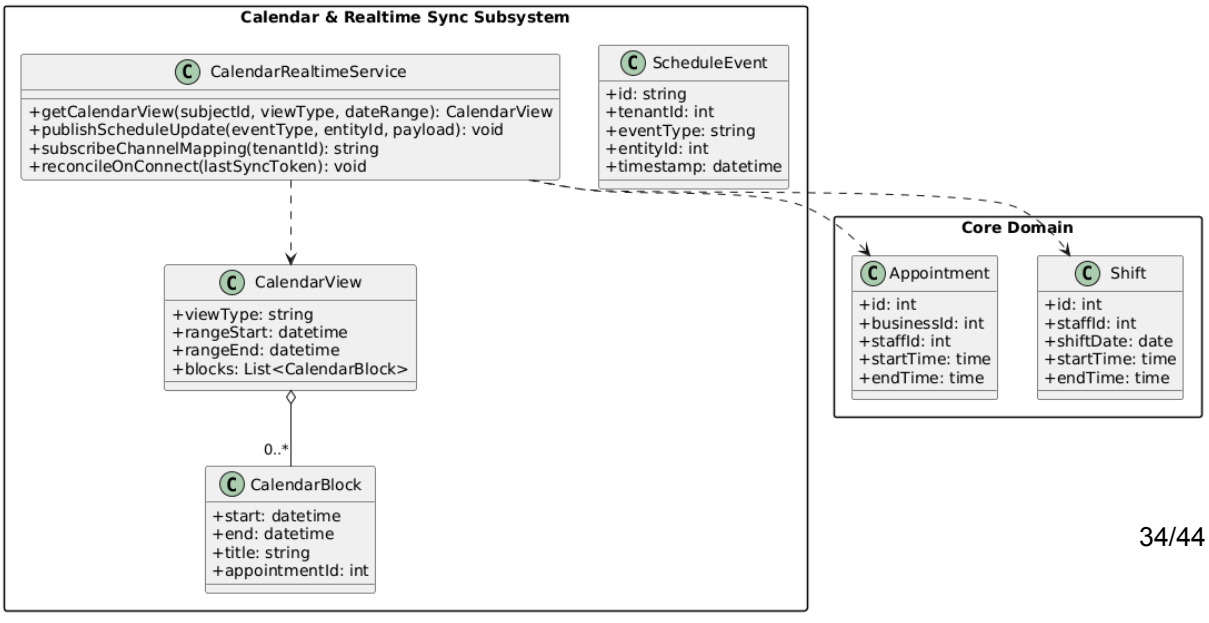
- **RealtimeSyncService.reconcileOnConnect(clientLastSyncToken?): SyncResult (optional)**
  On reconnect, compares the client's last sync token against server-side event history/checkpoints and returns missed updates (or a full refresh directive) to recover from temporary disconnects without corrupting client state.

Pre/Postconditions & Error cases:

- **Pre:**
  - Tenant scope must be validated (tenantId must match the entityId scope).
  - Request payload and event payload schema must be valid.
  - Firebase credentials/config must be available and realtime write permissions must match tenant isolation rules.

- **Post:**
  - Calendar view is returned in a deterministic, UI-friendly format for the requested range/viewType.
  - Realtime updates propagate to authorized clients within target latency (≤ 1s).
  - Client state remains consistent with backend source-of-truth via tokens/checkpoints.

- **Errors:**
  - **RealtimeWriteFailed** (log + retry with backoff)
  - **StaleSyncToken** (force reconcile/full refresh)
  - **UnauthorizedChannelAccess** (tenant mismatch or invalid scope)
  - **InvalidViewType / InvalidDateRange** (request validation)



4.6 Notification Subsystem (Future Extension)

Responsibility:
Generates and delivers email notifications (confirmation, reminder, cancellation) for appointment events. Runs asynchronously to avoid blocking core request/response flows and logs failures for retries.

Provided Services / Operations

- **NotificationService.sendConfirmationEmail(appointmentId, recipients): DeliveryResult**
  Builds and sends the appointment confirmation email to the provided recipients. Fetches appointment details, renders the appropriate localized template, dispatches via the configured email provider, and records delivery outcome.

- **NotificationService.sendReminderEmail(appointmentId, scheduledTime, recipients): DeliveryResult**
  Schedules and/or sends a reminder email for a future time (scheduledTime). Ensures idempotency (prevents duplicate reminders), renders localized content, and logs the final send status.

- **NotificationService.sendCancellationEmail(appointmentId, recipients): DeliveryResult**
  Builds and sends the cancellation email to recipients. Includes cancellation reason/context (if available), renders localized content, dispatches, and records outcome for audit/troubleshooting.

- **NotificationQueue.queueNotification(type, payload, priority): QueueAck**
  Enqueues a notification job for async delivery (e.g., CONFIRMATION, REMINDER, CANCELLATION). Validates payload schema, assigns priority, stores a job record, and returns an acknowledgement/jobId.

- **NotificationQueue.retryFailedNotifications(policy): RetryReport**
  Retries failed jobs according to a retry policy (maxAttempts, backoff, dead-letter rules). Updates job state transitions (FAILED → RETRYING → SENT / DEAD) and emits operational logs/metrics.

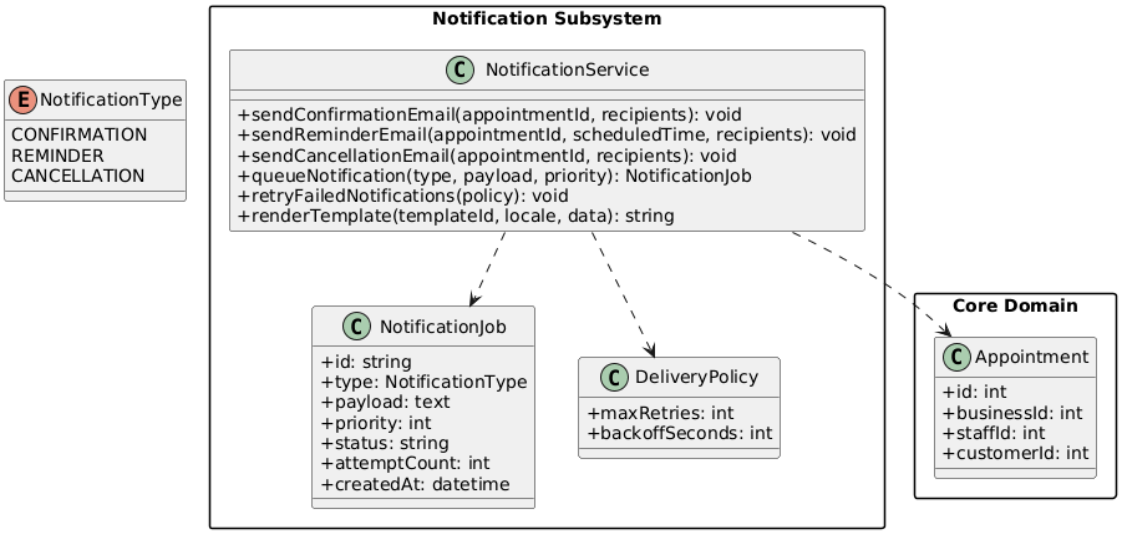
- **TemplateService.renderTemplate(templateId, locale, data): RenderedContent**
  Produces localized email subject/body from templates using the provided data model. Supports per-tenant overrides (optional) and ensures safe rendering (escaping, required fields validation).

Pre/Postconditions & Error cases:

- **Pre:**
  - Appointment must exist and be accessible under validated tenant scope.
  - Recipients must have valid email addresses and notification preferences must allow sending (if preferences are implemented).
  - TemplateId/locale must resolve to an available template and required data fields must be present.
  - Email provider configuration/credentials must be available.

- **Post:**
  - Notification job and delivery status are logged/audited (job state, timestamps, provider messageId if available).
  - Failures are persisted for retry (or dead-letter) according to policy.
  - Core appointment flows remain non-blocking (async dispatch/queue-based execution).

- **Errors:**
  - **EmailProviderUnavailable** (log + retry/backoff)
  - **InvalidRecipient** (reject job or mark as failed without retry depending on policy)
  - **TemplateRenderError** (mark failed; alert if recurring)
  - **NotificationJobNotFound / InvalidPayloadSchema** (validation/consistency)
  - **DuplicateNotification** (idempotency conflict, e.g., reminder already sent)



4.7 Search & Discovery Subsystem (Future Extension)

Responsibility:
Supports client-facing discovery flows: searching businesses, filtering by service, and selecting eligible staff before booking. Ensures only public/authorized data is exposed and applies query-level protections.

Provided Services / Operations

- **DiscoveryController.searchBusinesses(req): Response**
  Searches businesses by keyword (name/description/tags as configured) and applies filters (city, category, rating, openNow, priceRange, etc. if supported). Returns a paginated list of public business cards (safe fields only) optimized for discovery UI.

- **DiscoveryController.filterBusinessesByService(req): Response**
  Filters businesses that offer a specific service (serviceType | serviceId). Ensures results are constrained to public/active businesses and returns paginated public business summaries plus service metadata needed for booking.

- **DiscoveryController.listBusinessPublicProfile(req): Response**
  Returns the public business profile for a given businessId (display info, address region, working hours/availability summary, public services, public staff roster if allowed). Excludes private/internal fields (ownerId, internal notes, private contacts, tenant keys).

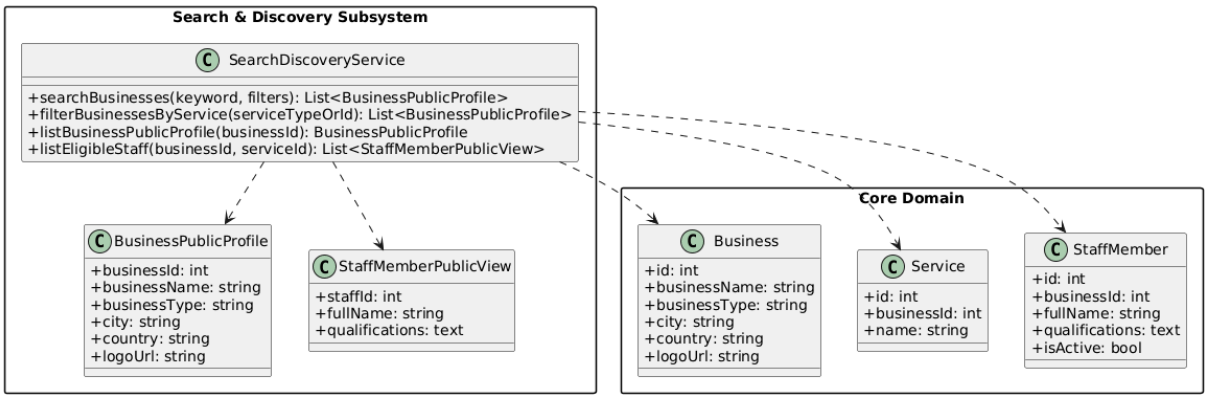
- **DiscoveryController.listEligibleStaff(req): Response**
  Returns staff eligible to deliver a selected service within a business context (businessId + serviceId). Applies staff-service mapping constraints, staff status (active), and optionally availability windows (if integrated with calendar subsystem).

Pre/Postconditions & Error cases:

- **Pre:**
  - Public exposure rules must be enforced (only public business/staff/service fields returned).
  - Query validation must be applied (max keyword length, allowed filters, pagination bounds).
  - Rate limiting must be enabled to prevent scraping/abuse (per IP/user/session).
  - Tenant isolation rules must be respected (no cross-tenant data returned even under shared indexes).

- **Post:**
  - Results are paginated, consistent, and contain only public/authorized data.
  - No cross-tenant leakage occurs in list/detail endpoints.
  - Query execution is protected (bounded filters, safe sorting, indexed lookups where possible).

- **Errors:**
  - **InvalidQuery** (unsupported filters, invalid pagination, malformed keyword)
  - **RateLimitExceeded** (throttle response, log abuse signal)
  - **BusinessNotFound / ServiceNotFound** (for profile/eligibility lookups)
  - **UnauthorizedPublicAccess** (if a resource is not publicly discoverable)



4.8 Admin Review & Platform Operations Subsystem (Future Extension)

Responsibility:
Implements superuser workflows and platform-wide operational controls. Handles business registration review/approval/rejection and (optionally) operational monitoring/configuration policies.

Provided Services / Operations

- **AdminReviewController.listPendingBusinessRegistrations(req): Response**
  Returns a paginated list of businesses awaiting review. Includes review-relevant fields only (business identity, submitted metadata, owner reference, submission timestamp, current state).

- **AdminReviewController.approveBusiness(req): Response**
  Approves and activates a business (businessId). Transitions business state from PENDING → APPROVED/ACTIVE, enables platform access (e.g., discovery visibility, staff onboarding eligibility), and writes an audit entry.

- **AdminReviewController.rejectBusiness(req): Response**
  Rejects a business (businessId) with a recorded reason. Transitions business state from PENDING → REJECTED, persists rejection reason + reviewer metadata, and triggers a notification workflow if configured.

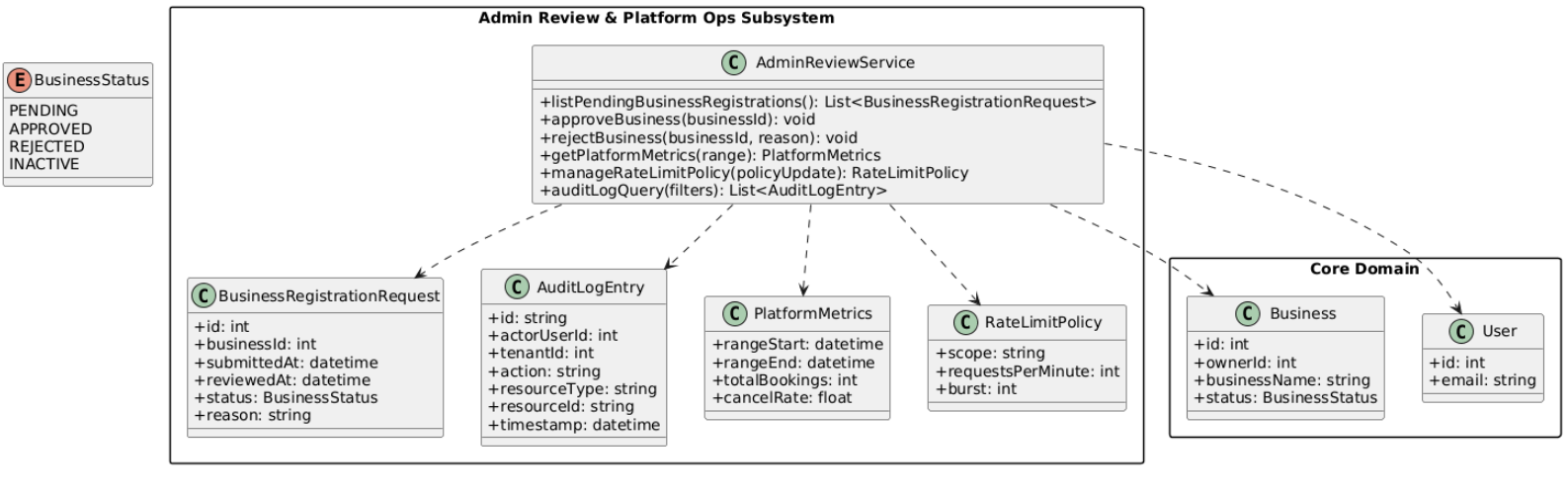
- **PlatformOpsController.getPlatformMetrics(req): Response (optional)**
  Queries basic operational metrics for a given range (e.g., new registrations, active businesses, booking volume, error rates). Returns aggregated, non-PII metrics only.

- **PlatformOpsController.manageRateLimitPolicy(req): Response (optional)**
  Updates rate limit configuration (policyUpdate) used by the API gateway/middleware. Validates policy schema, applies versioning, and logs changes for traceability.

- **AuditLogController.auditLogQuery(req): Response (optional)**
  Queries audit logs using filters (actor, entityType, entityId, action, dateRange). Returns paginated results suitable for investigations and compliance review.

Pre/Postconditions & Error cases:

- **Pre:**
  - Admin role required (RBAC: ADMIN/SUPERUSER).
  - Target business must exist and be in a valid state for transition.
  - All actions must be audit-logged (who/when/what, before/after state, reason if rejection).
  - Optional ops endpoints require additional safeguards (e.g., elevated permission, MFA, IP allowlist) if implemented.
- **Post:**
  - Approved businesses enter operational workflows (can be activated/visible and proceed with onboarding).
  - Rejections are recorded with reason and reviewer metadata; communication/notification is triggered if configured.
  - Platform policy changes (rate limits) and investigations (audit queries) are traceable and reproducible.
- **Errors:**
  - **Unauthorized** (missing/insufficient role)
  - **InvalidBusinessState** (cannot approve/reject from current state)
  - **AlreadyApproved/Rejected** (idempotency / state conflict)
  - **BusinessNotFound** (invalid businessId)
  - **PolicyValidationError** (for rate limit updates)



5 Glossary of Terms

Term	*Definition*
Rendivo	The multi-tenant appointment management platform described in this document.
SDD	Software Design Document that defines the architectural structure and design decisions of the Rendivo system.
User	A registered account within the Rendivo system representing an individual actor interacting with the platform.
Customer	A user role representing an individual who creates and manages appointments with a business.
Staff Member	A user role representing an employee of a business who provides services and participates in appointments.
Business Owner	A user role responsible for managing a business, its staff members, services, and schedules.
Business (Tenant)	An independent organizational entity operating within the Rendivo platform under the multi-tenant architecture.
Multi-Tenancy	An architectural approach where multiple businesses share the same system infrastructure while maintaining strict data isolation.
Tenant Isolation	The enforcement of data and access boundaries to ensure that one business cannot access another business’s data.
RBAC	Role-Based Access Control mechanism that restricts system operations based on user roles.
JWT	JSON Web Token used for stateless authentication and authorization between clients and the backend system.
Authentication	The process of verifying a user’s identity before granting access to the system.
Authorization	The process of determining whether an authenticated user has permission to perform a specific operation.
Appointment	A scheduled time interval that represents a service booking between a customer and a staff member.
Shift	A predefined time interval that represents the availability of a staff member for appointments.
Availability	The set of time intervals during which a staff member can accept appointments.

Term	*Definition*
Service	A business-defined offering that can be booked by customers through appointments.
State-Based Control	A control mechanism in which system behavior is governed by predefined states and valid state transitions.
Appointment State	The lifecycle status of an appointment (Pending, Confirmed, Completed, Cancelled).
REST API	A stateless application programming interface that exposes system functionality over HTTP using REST principles.
Persistence Layer	The system layer responsible for storing and retrieving data from the relational database.
Relational Database	A structured data storage system used to maintain appointments, users, businesses, and related entities with integrity constraints.

6 Traceability

This section establishes traceability between the functional and nonfunctional requirements defined in the Requirements Analysis Document and the architectural elements described in this Software Design Document. The purpose of this mapping is to demonstrate that each requirement is explicitly addressed by one or more design components, subsystems, or control mechanisms, ensuring completeness, consistency, and verifiability of the proposed architecture.

6.1 Functional Requirements Traceability

Requirement ID	Requirement Description Summary	Related SDD Section(s)
FR-1, FR-2	Business registration and unique business ID	3.2 Subsystem Decomposition, 3.4 Persistent Data Management
FR-3, FR-4	Business profile creation and update	3.2 Subsystem Decomposition, 3.6 Global Software Control
FR-5	Staff registration using business ID	3.2 Subsystem Decomposition, 3.7.4 User and Business Constraints
FR-6	Support for multiple user roles	1.2 Design Goals, 3.5 Access Control and Security
FR-7	JWT-based role-permission enforcement	3.5 Access Control and Security, 3.6.2 Authentication and Authorization Control
FR-8	Separate login interfaces per role	3.6.1 Control Architecture
FR-9–FR-11	Superuser approval and rejection of businesses	3.2 Subsystem Decomposition, 3.6 Global Software Control
FR-12–FR-16	Staff record management and attributes	3.2 Subsystem Decomposition, 3.4 Persistent Data Management
FR-17–FR-18	Staff shift scheduling constraints	3.6.4 State-Based Control, 3.7.3 Scheduling and Appointment Constraints

Requirement ID	Category	Related SDD Section(s)
FR-19	Staff dashboard after login	3.6.1 Control Architecture
FR-20–FR-24	Service management (CRUD, duration, price)	3.2 Subsystem Decomposition, 3.4 Persistent Data Management
FR-25–FR-28	Business, service, and staff selection	3.6.3 Appointment Control Flow
FR-29–FR-35	Appointment creation, modification, cancellation	3.6.3 Appointment Control Flow, 3.6.4 State-Based Control
FR-36	Appointment cancellation notifications	3.6.5 Integration with External and Asynchronous Services
FR-37–FR-38	Booking date and availability constraints	3.7.3 Scheduling and Appointment Constraints
FR-39	Calendar-based appointment visualization	3.6 Global Software Control
FR-40	Real-time schedule synchronization	3.6.5 Integration with External and Asynchronous Services
FR-41–FR-43	Email notifications	3.6.5 Integration with External and Asynchronous Services
FR-44–FR-45	Client registration and dashboard	3.2 Subsystem Decomposition, 3.6.1 Control Architecture
FR-46	Cancellation behavior summaries (optional)	3.4 Persistent Data Management
FR-47	RESTful Node.js/Express APIs	3.6.1 Control Architecture
FR-48	Tenant-based data isolation	1.2 Design Goals, 3.7.4 User and Business Constraints
FR-49	User-facing error messages	3.7.5 Error Handling and Failure Conditions

6.2 Nonfunctional Requirements Traceability

Requirement ID	Category	Related SDD Section(s)
NFR-1	Usability	3.6.1 Control Architecture
NFR-2	Reliability	3.6.4 State-Based Control, 3.7.3 Scheduling Constraints
NFR-3	Reliability (Optional)	3.6.4 State-Based Control
NFR-4	Performance	3.6 Global Software Control
NFR-5	Performance	3.6.5 Integration with External and Asynchronous Services
NFR-6	Performance	3.6.1 Control Architecture
NFR-7	Supportability	3.4 Persistent Data Management
NFR-8	Implementation	3.4 Persistent Data Management
NFR-9	Implementation	3.4 Persistent Data Management
NFR-10	Security	3.5 Access Control and Security
NFR-11	Interface	3.6.1 Control Architecture
NFR-12	Packaging	Deployment and Client Architecture (Section 3)
NFR-13	Environment Management	3.3 Hardware/Software Mapping
NFR-14	Legal / Security	3.7.6 Security Boundaries
NFR-15	Legal / Security	3.5 Access Control and Security